实验报告

（一）实验题目

图卷积神经网络的实现

（二） 实验目的

1. 掌握图数据结构的存储与操作。
2. 理解并实现图卷积神经网络（GCN）的基本原理与操作。
3. 通过图卷积网络了解图数据的信息传播过程，并提高程序设计及数据结构应用能力。

（三）实验要求

1. 自定义图的结点数量和图邻接矩阵中保存的连接关系。
2. 实现图卷积神经网络时，只需要实现前向传播，即能够更新结点的特征即可。如果可以实现反向传播，可作为加分项。
3. 可参考附录中关于图神经网络的介绍。

（四） 实验内容；

1. 图存储

- 采用邻接矩阵（或邻接表）表示图，掌握基本的图操作（添加/删除边或节点）。

2. 图连通性判断

- 实现广度优先搜索（BFS）判断图的连通性。

3. 稀疏矩阵乘法

- 针对图数据稀疏性的特点，实现稀疏矩阵乘法算法，优化图卷积计算效率。

4. 图卷积神经网络（GCN）实现

- 实现一个简化版的 GCN，通过归一化邻接矩阵和节点特征矩阵对节点特征进行更新，并利用 ReLU 函数引入非线性。要求矩阵乘法是以稀疏矩阵形式。

（五） 实验代码

```c
#define MAX_NODES 100
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

// 图结构体与基本操作
typedef struct
{
    int num_nodes;//节点数
    int adj[MAX_NODES][MAX_NODES]; // 邻接矩阵 ，adj[i][j]=1 表示 i 和 j 有边
} Graph;

// 稀疏矩阵结构体
// 用三元组(row, col, value)存储稀疏矩阵的非零元素
typedef struct
{
    int row, col;
    float value;//非零元素的值
} SparseEntry;

typedef struct
{
    int rows, cols, nnz;//行数，列数，非零元素个数
    SparseEntry entries[MAX_NODES * MAX_NODES];//存储非零元素
} SparseMatrix;

// 初始化图
void init_graph(Graph* g, int n)
{
    g->num_nodes = n;//设置节点数
    memset(g->adj, 0, sizeof(g->adj));//邻接矩阵清零
}
```

```c
// 添加无向边
void add_edge(Graph* g, int u, int v)
{
    g->adj[u][v] = 1;
    g->adj[v][u] = 1; // 无向图
}

// 删除无向边
void remove_edge(Graph* g, int u, int v)
{
    g->adj[u][v] = 0;
    g->adj[v][u] = 0;
}

// 打印邻接矩阵
void print_adj_matrix(Graph* g)
{
    printf("邻接矩阵：\n");
    for (int i = 0; i < g->num_nodes; i++)
    {
        for (int j = 0; j < g->num_nodes; j++)
        {
            printf("%d ", g->adj[i][j]);
        }
        printf("\n");
    }
}

// 图连通性判断（BFS）
// 使用广度优先搜索遍历图，visited 数组标记访问过的节点
void bfs(Graph* g, int start, int* visited)
{
    int queue[MAX_NODES], front = 0, rear = 0; // 队列，队头指针，队尾指针
    queue[rear++] = start; // 起始节点入队
    visited[start] = 1;    // 起始节点已访问
    while (front < rear)
    {
        int u = queue[front++]; // 出队一个节点 u
        for (int v = 0; v < g->num_nodes; v++) // 遍历所有节点，检查与 u 相连的节点 v
        {
            if (g->adj[u][v] && !visited[v])  // 如果 u 和 v 有边且 v 未被访问
            {
                visited[v] = 1;      // 标记 v 已访问
                queue[rear++] = v;   // v 入队，等待后续遍历
```

```
            }
        }
    }
}

// 判断图是否连通：从 0 号节点出发，若所有节点都被访问则连通
int is_connected(Graph* g)
{
    int visited[MAX_NODES] = {0};//标记访问过的节点
    bfs(g, 0, visited);//从 0 号节点出发
    for (int i = 0; i < g->num_nodes; i++)
    {
        if (!visited[i]) return 0;
    }
    return 1;//连通
}

// 将邻接矩阵转为稀疏矩阵存储
void dense_to_sparse(Graph* g, SparseMatrix* sm) // sm: 输出的稀疏矩阵结构体
{
    sm->rows = g->num_nodes; // 设置稀疏矩阵的行、列数
    sm->cols = g->num_nodes;
    sm->nnz = 0; // 初始化非零元素个数为0
    for (int i = 0; i < g->num_nodes; i++)// 遍历邻接矩阵的每个元素
    {
        for (int j = 0; j < g->num_nodes; j++)
        {
            if (g->adj[i][j] != 0)// 如果邻接矩阵(i, j)位置有边（非零）
            {
                // 记录该非零元素的位置和值到稀疏矩阵中
                sm->entries[sm->nnz].row = i;
                sm->entries[sm->nnz].col = j;
                sm->entries[sm->nnz].value = (float)g->adj[i][j];
                sm->nnz++; // 非零元素个数加1
            }
        }
    }
}

// 稀疏矩阵与稠密矩阵乘法 Y = A * X
// A: 稀疏矩阵 (rows x cols), X: (cols x X_cols), Y: (rows x X_cols)
void sparse_dense_mul(SparseMatrix* A, float* X, float* Y, int X_cols)
{
```

```
    memset(Y, 0, sizeof(float) * A->rows * X_cols);// 先将结果矩阵Y清零，Y 的大小为A->rows
行，X_cols 列
    for (int k = 0; k < A->nnz; k++)
    {
        int i = A->entries[k].row;
        int j = A->entries[k].col;
        float val = A->entries[k].value;
        // 对于X 的每一列，累加到Y 的对应位置
        for (int c = 0; c < X_cols; c++)
        {
            Y[i * X_cols + c] += val * X[j * X_cols + c];// Y[i][c] += val * X[j][c]
        }
    }
}

// 添加自环：邻接矩阵对角线元素设为1
void add_self_loop(Graph* g)
{
    for (int i = 0; i < g->num_nodes; i++)
    {
        g->adj[i][i] = 1;
    }
}

// 计算度矩阵D（只存储对角线元素）
//统计每个节点的度数，结果存入D
void calc_degree(Graph* g, float* D)
{
    for (int i = 0; i < g->num_nodes; i++)
    {
        int sum = 0; // 用于累计第i 个节点的度数
        for (int j = 0; j < g->num_nodes; j++)
        {
            sum += g->adj[i][j]; // 如果有边则加1
        }
        D[i] = (float)sum; // 存储第i 个节点的度数
    }
}

// 归一化邻接矩阵 A'' = D^{-1/2} * A' * D^{-1/2}
// 归一化后，邻接矩阵每个元素A''[i][j] = A'[i][j] / sqrt(D[i]*D[j])
void normalize_adj(Graph* g, float norm_adj[MAX_NODES][MAX_NODES])
{
    float D[MAX_NODES];
```

```
    calc_degree(g, D);//计算每个结点的度
    for (int i = 0; i < g->num_nodes; i++)
    {
        for (int j = 0; j < g->num_nodes; j++)
        {
            if (g->adj[i][j])
            {
                norm_adj[i][j] = g->adj[i][j] / sqrtf(D[i] * D[j]);
            }
            else
            {
                norm_adj[i][j] = 0.0f;
            }
        }
    }
}

// 稠密邻接矩阵转稀疏矩阵（float 型）
void densef_to_sparsef(float adj[MAX_NODES][MAX_NODES], int n, SparseMatrix* sm)
{
    //初始化
    sm->rows = n;
    sm->cols = n;
    sm->nnz = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (fabs(adj[i][j]) > 1e-6) // 如果邻接矩阵(i, j)位置有边（非零），防止误差，设置
一个很小的阈值
            {
                sm->entries[sm->nnz].row = i;// 记录该非零元素的位置和值到稀疏矩阵中
                sm->entries[sm->nnz].col = j;
                sm->entries[sm->nnz].value = adj[i][j];
                sm->nnz++;
            }
        }
    }
}

// ReLU 激活函数，对每个元素 x，若 x<0 则置 0，否则不变
void relu(float* X, int n)
{
    for (int i = 0; i < n; i++)
```

```
    {
        if (X[i] < 0) X[i] = 0;
    }
}

// 打印特征矩阵，每行对应一个节点的特征
void print_features(float* X, int n, int d)
{
    for (int i = 0; i < n; i++)
    {
        printf("节点%d: ", i);
        for (int j = 0; j < d; j++)
        {
            printf("%.4f ", X[i * d + j]);
        }
        printf("\n");
    }
}

int main()
{
    int n = 4; // 节点数
    int feature_dim = 3; // 输入特征维度
    int out_dim = 2; // 输出特征维度
    Graph g;
    //初始化图
    init_graph(&g, n);
```

```
    // 构造一个简单的无向环图
    add_edge(&g, 0, 1);
    add_edge(&g, 1, 2);
    add_edge(&g, 2, 3);
    add_edge(&g, 3, 0);
    //打印邻接矩阵
    print_adj_matrix(&g);
    //判断图是否连通
    printf("图%s 连通\n", is_connected(&g) ? "是" : "否");

    // 步骤1：添加自环
    add_self_loop(&g);
    printf("加自环后邻接矩阵：\n");
    print_adj_matrix(&g);//打印邻接矩阵

    // 步骤2：归一化邻接矩阵
```

```
    float norm_adj[MAX_NODES][MAX_NODES];
    normalize_adj(&g, norm_adj);

    // 步骤3：初始化节点特征矩阵X (n x feature_dim)
    float X[MAX_NODES * feature_dim];
    for (int i = 0; i < n * feature_dim; i++)
    {
        X[i] = (float)(rand() % 10) / 10.0f; // 随机初始化
    }
    printf("初始特征：\n");
    print_features(X, n, feature_dim);

    // 步骤4：初始化权重矩阵W (feature_dim x out_dim)
    float W[feature_dim * out_dim];
    for (int i = 0; i < feature_dim * out_dim; i++)
    {
        W[i] = (float)(rand() % 10) / 10.0f;//随机初始化
    }

    // 步骤5：将归一化邻接矩阵转为稀疏矩阵
    SparseMatrix normA_sparse;
    densef_to_sparsef(norm_adj, n, &normA_sparse);

    // 步骤6：A'' * X，信息聚合
    float AX[MAX_NODES * feature_dim];
    sparse_dense_mul(&normA_sparse, X, AX, feature_dim);

    // 步骤7：(A''X) * W，特征线性变换
    float H[MAX_NODES * out_dim];
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < out_dim; j++)
        {
            H[i * out_dim + j] = 0;//初始化
            for (int k = 0; k < feature_dim; k++)
            {
                H[i * out_dim + j] += AX[i * feature_dim + k] * W[k * out_dim + j];//H[i][j]
= AX[i][k] * W[k][j]
            }
        }
    }
    // 步骤8：ReLU 激活，得到最终特征
    relu(H, n * out_dim);
    printf("GCN 更新后特征：\n");
```

```
    print_features(H, n, out_dim);


    return 0;
}
```

（六） 实验结果

```
    //构造的无向环图为如下：
    add_edge(&g, 0, 1);
    add_edge(&g, 1, 2);
    add_edge(&g, 2, 3);
    add_edge(&g, 3, 0);
```

